```
print(dict1)
# Delete a single element
del dict1['a']
 print("post delete key dict1:", dict1)
# Delete all elements in the dictionary
dict1.clear()
print("post clear dict1: ", dict1)
dict2 = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
# Delete the dictionary
del dict2
 print("complete dict delete dict2: ", dict2)

Output:
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
post delete key dict1: {'b': 2, 'c': 3, 'd': 4, 'e': 5}
post clear dict1:  {}
Traceback (most recent call last):
  File "dict_sample.py", line 134, in <module>
    print("complete dict delete dict2: ", dict2)
NameError: name 'dict2' is not defined
```

Fromkeys

The fromkeys() method returns a dictionary with the specified keys and the specified value. It creates a new dictionary from the given sequence with the specific value, else creates with the *None* value assigned. For example:

```
types = ('flower', 'diamond', 'heart')
cards = dict.fromkeys(types)
print("Create a dict with None values:", cards)
qty = [10]
cards = dict.fromkeys(types, qty)
print("Create a dict with default values:", cards)
qty.append(3)
print("View the created dict with updated list values:",
cards)
qty.pop(0)
print("View the created dict with updated list values:",
cards)

Output:
Create a dict with None values: {'flower': None, 'diamond':
None, 'heart': None}
Create a dict with default values: {'flower': [10], 'diamond':
[10], 'heart': [10]}
View the created dict with updated list values: {'flower':
[10, 3], 'diamond': [10, 3], 'heart': [10, 3]}
View the created dict with updated list values: {'flower':
[3], 'diamond': [3], 'heart': [3]}
```

Setdefault

setdefault() returns the value of a key (if the key is in dictionary). Else, it inserts a key with the default value to the dictionary. It increases speed significantly by avoiding redundant key lookups. For example:

```
d = {'a': 97, 'b': 98, 'c': 99, 'd': 100}
ret_value = d.setdefault('e', 120)
print("New key which does not exists in the dict, value: {},
dict: {}", ret_value, d)
ret_value = d.setdefault('e', 140)
print("New key which exists in the dict, value: {}, dict:
{}", ret_value, d)

Output:
New key which does not exists in the dict, value: {}, dict:
{} 120 {'a': 97, 'b': 98, 'c': 99, 'd': 100, 'e': 120}
New key which exists in the dict, value: {}, dict: {} 120
{'a': 97, 'b': 98, 'c': 99, 'd': 100, 'e': 120}
```

Defaultdict

A defaultdict is configured to create items on demand whenever a missing key is searched. Internally it works by calling the callable provided to produce a default value.

The callable that produces the default values is held in an instance attribute called default_factory. Here's an example:

- Create a defaultdict with a list constructor as default_ factory:

```
index = collections.defaultdict(list)
```

- If the key is not found, an empty list is assigned to the new key.
- If no default_factory is provided, the usual KeyError is raised for the missing keys.

Dict comprehensions

You can transform a dictionary from one form to another using dict comprehensions. Dict comprehensions make the code easier to read, avoid loops and enhance performance *(https://stackoverflow.com/questions/52542742/why-is-this-loop-faster-than-a-dictionary-comprehension-for-creating-a-dictionary)*. Here's an example:

```
dict1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
# Double each value in the dictionary
double_dict1 = {k:v*2 for (k,v) in dict1.items()}
print(double_dict1)

Output:
{'a': 2, 'b': 4, 'c': 6, 'd': 8, 'e': 10}
```

You can log on to *https://www.datacamp.com/tutorial/python-dictionary-comprehension* for multiple such examples.